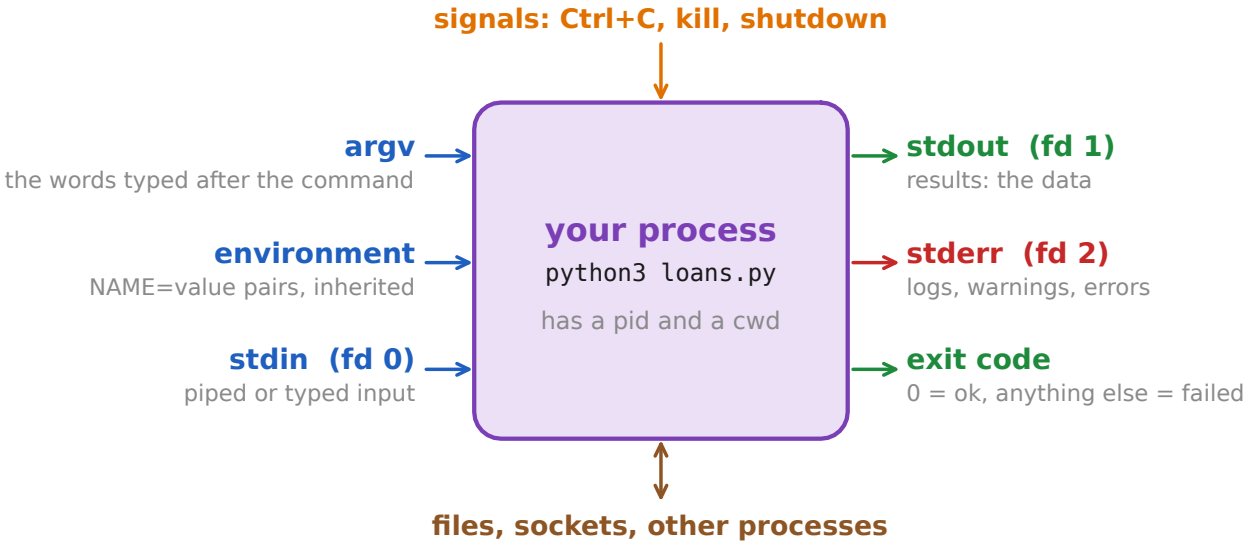
Chapter 14

The OS and the command line

every program talks to the OS the same way

To the operating system, every program in every language is a **process** with the same few connections. Learn them once and you can drive any program from Python, and write Python tools that behave like real Unix commands:



A shell pipeline like `grep ERROR app.log | sort | uniq -c > counts.txt` works only because every program in it follows these conventions: read stdin, write results to stdout, complain on stderr, report success with the exit code. This chapter makes your Python programs good citizens of that world.

sys and os: the process from the inside

`sys` is about the **interpreter** (its arguments, streams, version, exit); `os` is about the **operating system** (environment, files, process ids). Running a child Python is the easiest way to see argv and exit codes for real:

```
1 import subprocess, sys
2
3 child = "import sys; print(sys.argv[1:]); sys.exit(3)"
4 r = subprocess.run([sys.executable, "-c", child, "a b", "c"], capture_output=True, text=True)
5 print(r.returncode, r.stdout.strip())
6
7 r = subprocess.run([sys.executable, "-c", "raise SystemExit('bad input')"],
8                     capture_output=True, text=True)
9 print(r.returncode, repr(r.stdout), r.stderr.strip())
```

```
3 ['a b', 'c']
1 '' bad input
```

line 4: "a b" stays one argument

line 7: SystemExit(str): message to stderr, exit code 1

exit code	means	who uses it
`0`	success	everything: `&&` in the shell, CI, cron, systemd
`1`	general failure	`sys.exit("message")`, an uncaught exception
`2`	you used it wrong (bad arguments)	argparse, grep, most Unix tools
`126` / `127`	found but not executable / not found	the shell
`128 + n`	killed by signal n (`130` = Ctrl+C, `137` = SIGKILL)	the shell

```
1 import os, subprocess, sys
2 from pathlib import Path
3
4 print(sys.platform, sys.version_info[:2], os.path.basename(sys.executable))
5 print(os.getcwd() == str(Path.cwd()), os.getpid() > 1, os.cpu_count() >= 1)
6 print(os.sep, repr(os.pathsep), sys.stdout.isatty())
7 print(os.getenv("LOANS_DB", "loans.db"))           # environment variable, with a default
8
9 env = {**os.environ, "LOANS_DB": "test.db"}       # the child gets its own copy
10 r = subprocess.run([sys.executable, "-c", "import os; print(os.environ['LOANS_DB'])"],
11                     env=env, capture_output=True, text=True)
12 print(r.stdout.strip(), "LOANS_DB" in os.environ)
```

```
linux (3, 14) python3
True True True
/ ':' False
loans.db
test.db False
```

line 6: False: output is being captured

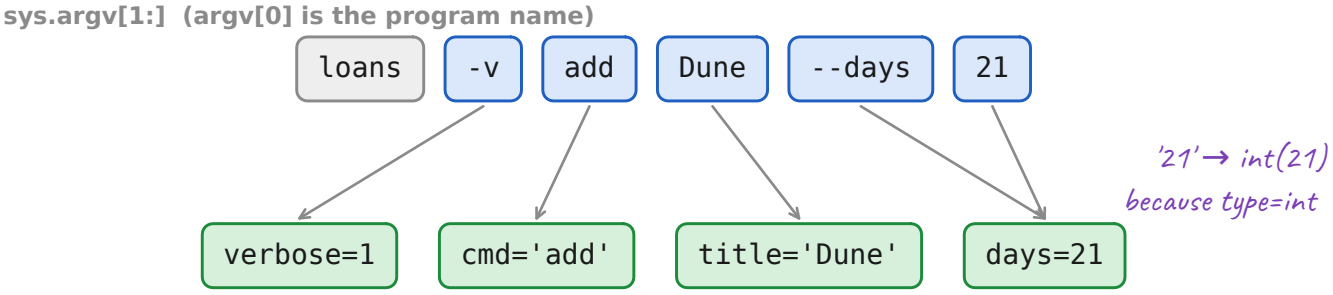
line 12: changes never flow back to the parent

TRAP

Environment variables are inherited by **every** child process, appear in crash reports, and can be read from `/proc` by the same user. They're the standard place for configuration and tolerable for secrets in containers, but never log `os.environ`, and don't pass secrets as command-line arguments: `ps` shows argv to every user on the machine.

argparse: a real command-line interface

`argparse` turns the list of words in `sys.argv` into a tidy object, converts types, checks choices, and writes `--help` and error messages for you:




```
1  import argparse
2
3  def make_parser():
4      p = argparse.ArgumentParser(prog="loans", description="Track library loans.")
5      p.add_argument("-v", "--verbose", action="count", default=0, help="-vv for more")
6      sub = p.add_subparsers(dest="cmd", required=True)
7      add = sub.add_parser("add", help="lend a book")
8      add.add_argument("title")
9      add.add_argument("--days", type=int, default=14, help="default: %(default)s")
10     ls = sub.add_parser("list", help="show loans")
11     ls.add_argument("--overdue", action="store_true")
12     ls.add_argument("--sort", choices=["title", "due"], default="due")
13     return p
14
15 print(make_parser().parse_args(["-v", "add", "Dune", "--days", "21"]))
16 print(make_parser().parse_args(["list", "--overdue"]))
```

```
Namespace(verbose=1, cmd='add', title='Dune', days=21)
Namespace(verbose=0, cmd='list', overdue=True, sort='due')
```

line 5: -v, -vv, -vvv

line 6: subcommands, like git commit

line 8: positional: required

line 9: option: has a default

line 11: a flag: True if present

```python
print(make_parser().format_help())
```

```
usage: loans [-h] [-v] {add,list} ...

Track library loans.

positional arguments:
  {add,list}
    add          lend a book
    list         show loans

options:
  -h, --help     show this help message and exit
  -v, --verbose  -vv for more
```

main(argv=None): the pattern that makes a CLI testable

```python
def main(argv=None):
    args = make_parser().parse_args(argv)        # None means: use sys.argv[1:]
    if args.cmd == "add":
        print(f"lent {args.title} for {args.days} days")
    return 0

print(main(["add", "Emma"]))                     # a test calls main() with a list
# in loans.py:   if __name__ == "__main__": sys.exit(main())
```

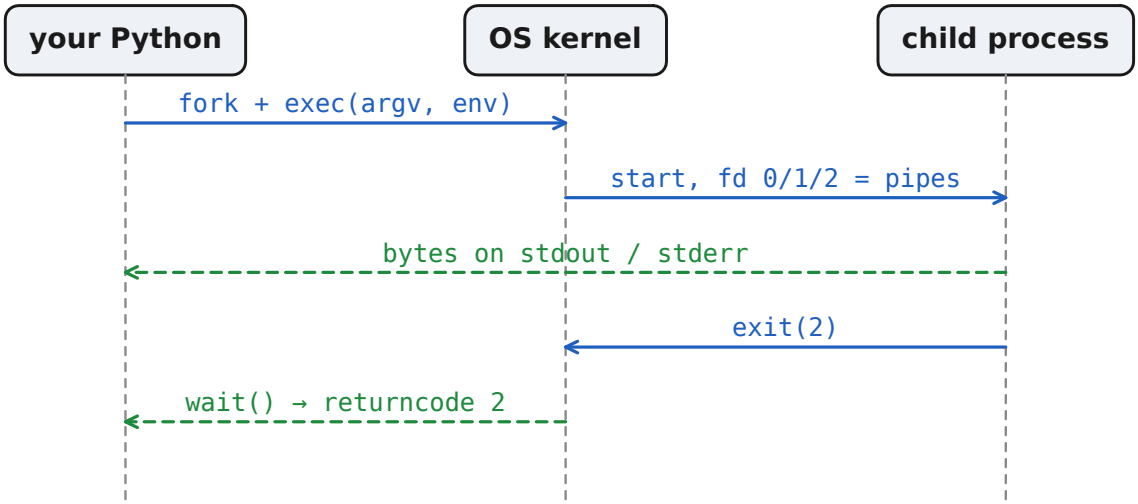
```
lent Emma for 14 days
0
```

the same program from the shell (usage errors go to stderr, exit code 2)

```
$ python3 loans.py add Dune --days 21
lent Dune for 21 days
$ echo $?
0
$ python3 loans.py list --sort author
usage: loans list [-h] [--overdue] [--sort {title,due}]
loans list: error: argument --sort: invalid choice: 'author' (choose from 'title', 'due')
$ echo $?
2
```

subprocess: running other programs

`subprocess.run` starts a program, waits for it, and hands back its exit code and output. Under the hood the OS creates a child process wired to pipes:




```
1  import subprocess, sys
2
3  try:
4      subprocess.run(["ls", "/no/such/dir"], capture_output=True, text=True, check=True)
5  except subprocess.CalledProcessError as e:
6      print("failed with", e.returncode, "|", e.stderr.strip())
7
8  try:
9      subprocess.run([sys.executable, "-c", "import time; time.sleep(5)"], timeout=0.2)
10 except subprocess.TimeoutExpired as e:
11     print("gave up after", e.timeout, "s (the child is killed)")
```

```
failed with 2 | ls: cannot access '/no/such/dir': No such file or directory
gave up after 0.2 s (the child is killed)
```

line 4: check=True: non-zero exit raises

line 9: always set a timeout for things that can hang

Never build a shell command from data

With `shell=True` your string goes to `/bin/sh`, which treats `;`, `|`, `$( )` and spaces as syntax. If any part came from a user or a filename, they can run anything. Here the "filename" smuggles in a second command (a harmless `echo`, but it could be `rm -rf ~`):

shell=True + f-string: injected

```
import subprocess
name = "notes.txt; echo HACKED"
r = subprocess.run(f"echo {name}", shell=True,
                   capture_output=True, text=True)
print(r.stdout)
```

```
notes.txt
HACKED
```

a list of arguments: safe

```
import subprocess
name = "notes.txt; echo HACKED"
r = subprocess.run(["echo", name],
                   capture_output=True, text=True)
print(r.stdout)
```

```
notes.txt; echo HACKED
```

RULE OF THUMB

Pass a **list**: each item reaches the program as exactly one argument and no shell ever parses it. If a value could start with `-`, put `"--"` before it so the program can't mistake it for an option. This is the same bug as SQL injection (Ch 12): data mixed into code.

Signals: being asked to stop

A signal is a tiny message from the OS: "stop", "the terminal closed", "reload". Ctrl+C sends `SIGINT`, which Python turns into `KeyboardInterrupt`. `kill`, `systemctl stop` and `docker stop` send `SIGTERM`, which by default kills Python on the spot: no `finally`, no flush. Handle it to shut down cleanly:

```
1  import subprocess, sys, textwrap
2
3  child = textwrap.dedent('''
4      import signal, sys, time
5      def stop(signum, frame):
6          print("got", signal.Signals(signum).name, "- saving and exiting")
7          sys.exit(0)                          # raises SystemExit: finally blocks run
8      signal.signal(signal.SIGTERM, stop)
9      print("ready", flush=True)
10     time.sleep(30)
11 ''')
12 p = subprocess.Popen([sys.executable, "-c", child], stdout=subprocess.PIPE, text=True)
13 print(p.stdout.readline().strip())
14 p.terminate()                                 # sends SIGTERM
15 out, _ = p.communicate(timeout=5)
16 print(out.strip(), "| exit code", p.returncode)
```

```
ready
got SIGTERM - saving and exiting | exit code 0
```

line 8: install the handler

line 12: Popen: start without waiting

signal	sent by	default	Python
SIGINT (2)	Ctrl+C	stop	raises KeyboardInterrupt
SIGTERM (15)	kill, systemd, Docker, Kubernetes	stop at once	handle it: clean up, exit
SIGKILL (9)	kill -9, the OOM killer	stop at once	**can't be caught**
SIGHUP (1)	terminal closed; daemons: "reload config"	stop	handle if you're a daemon
SIGPIPE (13)	reader of your pipe went away (| head)	stop	ignored: you get BrokenPipeError

HONEST NOTE

Windows has no SIGTERM in this sense: `terminate()` there kills the process outright. Only Ctrl+C (SIGINT) and SIGBREAK are portable. Signal handlers run only in the main thread, between bytecodes, so keep them tiny: set a flag or raise.

Logging, and the good-citizen checklist

A command-line tool's stdout is often piped into another program, so it must contain only results. Progress and diagnostics go to stderr through `logging`, with `-v` turning up the detail:

```
1  import io, logging
2
3  def setup_logging(verbose, stream):                    # pass sys.stderr in real life
4      log = logging.getLogger("loans")
5      log.setLevel([logging.WARNING, logging.INFO, logging.DEBUG][min(verbose, 2)])
6      handler = logging.StreamHandler(stream)
7      handler.setFormatter(logging.Formatter("%(levelname)s %(message)s"))
8      log.handlers[:] = [handler]
9      log.propagate = False
10     return log
11
12 for v in (0, 2):
13     buf = io.StringIO()
14     log = setup_logging(v, buf)
15     log.debug("opening loans.db"); log.info("3 loans found"); log.warning("1 overdue")
16     print(f"-v x{v}:", buf.getvalue().splitlines())
```

```
-v x0: ['WARNING 1 overdue']
-v x2: ['DEBUG opening loans.db', 'INFO 3 loans found', 'WARNING 1 overdue']
```

line 5: -v count picks the level

a good command-line program...	so that...
writes results to stdout, messages to stderr	`prog > out.txt` and pipes get clean data
exits 0 on success, non-zero on failure (2 for usage)	&&, scripts, CI and cron can react
reads stdin when given no file, or `-`	it works in the middle of a pipeline

has `--help` and clear error messages	nobody has to read the source (argparse does it)
takes config as flags > env vars > file > defaults	the same build runs anywhere (practice 14.4)
skips colour and progress bars if `not sys.stdout.isatty()`	logs and pipes stay readable
handles SIGTERM and Ctrl+C: cleans up, then exits	containers and services stop it safely

▢ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 14.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 14.1` shows other valid solutions and their trade-offs.

`14.1` build from a spec An argparse CLI from a spec
`14.2` fix the bug Close the shell-injection hole
`14.3` build from a spec A grep-like filter with proper exit codes
`14.4` build from a spec Config from flags, environment and defaults
`14.5` learn an unseen module Read a .env file with shlex
`14.6` mini-project A tiny task runner

PASSPORT STAMP ask the same question of every new language (Appendix B)

1. How does code run?

To the OS, a Python program is a process: argv and environment in, stdin/stdout/stderr, an exit code out, signals from outside. `python3 file.py` runs the interpreter, which runs your file.

10. I/O

Text streams `sys.stdin`/`stdout`/`stderr`; results to stdout, messages to stderr.

12. Stdlib and tooling

`sys`, `os`, `argparse`, `subprocess` (list args, `check`, `timeout`), `signal`, `logging`, `shlex`.

IN OTHER LANGUAGES

C: `int main(int argc, char **argv)`, the return value is the exit code. Java: `main(String[] args)`, `System.exit`, `ProcessBuilder`. Go: `os.Args`, `flag`, `os/exec`. Rust: `std::env::args`, `std::process::Command`, and the `clap` crate as its argparse. The process model underneath is identical.

DEEP DIVE (your vault)

Vault notes: 20.2 Programs as managed processes · 20.3 Exit status and supervision · 20.4 Configuration · 20.5 Filesystem and runtime conventions · 19.11 Command injection and subprocess boundaries · 19.15 Secrets · 12.12 Logging and diagnostics.